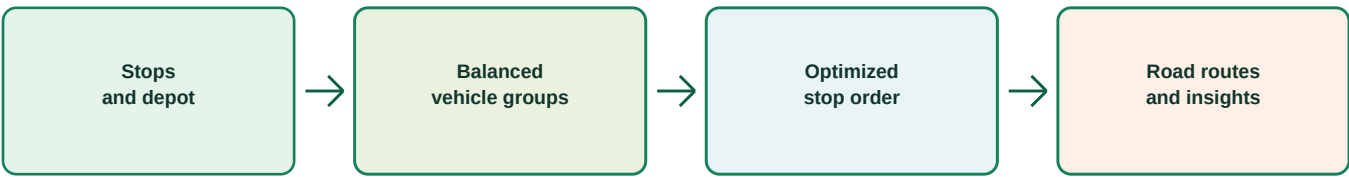


ROUTEMASTER

Intelligent Multi-Vehicle Route Planning & Optimization Platform



Technical Project Documentation

A complete guide to the product concept, architecture, diagrams, data flow, algorithms, modules, API, database, setup, and verification.

Document	RouteMaster Technical Documentation
Product type	Multi-vehicle route planning web application
Frontend	React, Vite, Leaflet, React-Leaflet, Axios
Backend	Python, Flask, SQLAlchemy, scikit-learn
Mapping	OpenStreetMap, Nominatim, OSRM
Storage	MySQL in configured environments; SQLite fallback for local demo
Prepared	2026-09-25

Contents

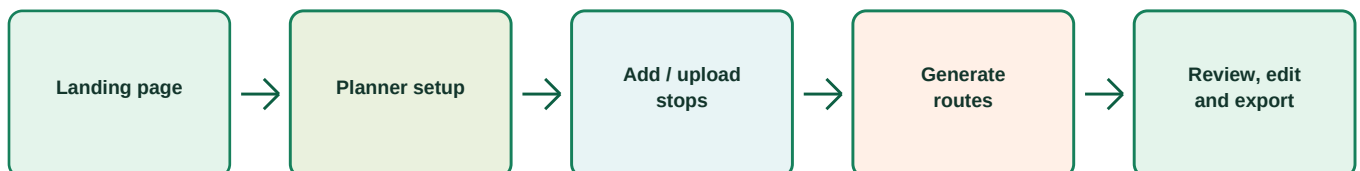
1. Product Overview
2. Architecture and Runtime Separation
3. End-to-End Route Generation Flow
4. Planner Experience and UI Modules
5. Location Input and Geocoding
6. Assignment and Balancing Algorithm
7. Route Sequencing and Road Routing
8. Backend Modules
9. REST API
10. Database Design
11. Frontend Structure
12. Export and History
13. Installation and Operation
14. Validation and Error Handling
15. Testing and Verification
16. Design Decisions and Extension Points

1. Product Overview

RouteMaster automatically distributes multiple locations across a user-selected number of vehicles and produces an optimized route for each vehicle. It is intended for delivery planning, field service, sales visits, maintenance teams, and other multi-stop operations.

The design separates location assignment from route sequencing. Assignment decides which vehicle serves each stop; sequencing decides the order in which that vehicle visits its assigned stops.

Primary user journey

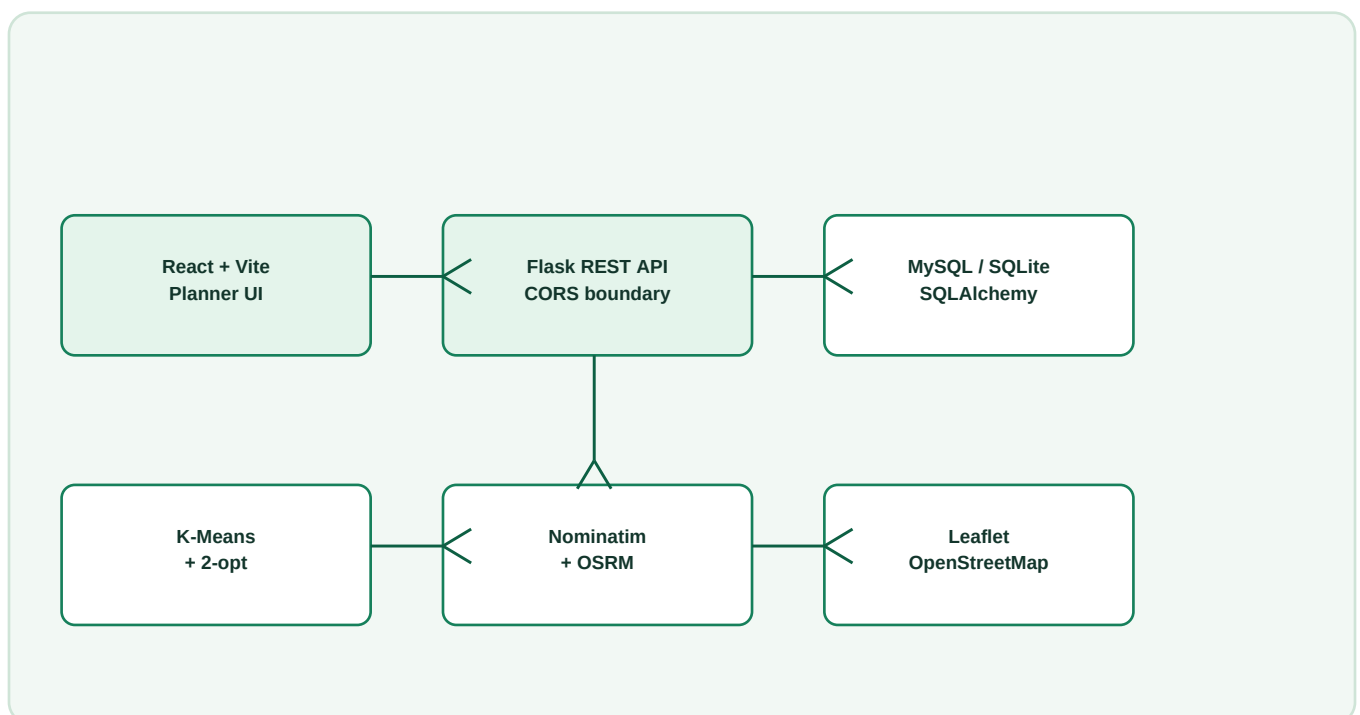


Product boundaries

- ? No login, signup, authentication, user profiles, payments, or subscriptions.
- ? No fuel-cost calculation and no claim to replace general-purpose navigation products.
- ? Geocoding and road routing are isolated behind backend services.
- ? The product focuses on planning, comparison, editing, history, and export.

2. Architecture and Runtime Separation

The frontend and backend are independent applications. React/Vite handles the user experience and Leaflet visualization. Flask handles validation, algorithms, routing-service integration, persistence, and JSON responses.



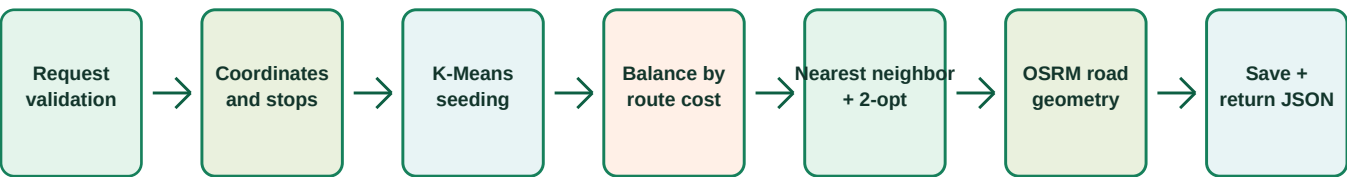
Runtime responsibilities

Layer	Responsibility	Runtime
Frontend	Pages, components, map, inputs, exports, loading and errors.	Vite on port 5173
Backend	REST API, clustering, optimization, routing requests, persistence.	Flask on port 5000
Database	Route plans and normalized location/route records.	MySQL or local SQLite
External services	Address search and road geometry.	Nominatim and OSRM over HTTPS

Configuration uses VITE_API_URL in the frontend and MYSQL_HOST, MYSQL_PORT, MYSQL_USER, MYSQL_PASSWORD, and MYSQL_DATABASE in the backend.

3. End-to-End Route Generation Flow

When the user clicks Generate Routes, the request follows this pipeline:



- ? The planner sends start, end, vehicle count, stops, and the optional maximum-stops value.
- ? The backend validates required fields and stop coordinates.
- ? K-Means creates geographic seeds; balancing adjusts stop counts and estimated route lengths.
- ? Each vehicle route is sequenced and improved before OSRM is called.
- ? The response includes route geometry, distance, duration, route order, and summary values.
- ? The generated plan is stored through SQLAlchemy and returned with a plan id.

4. Planner Experience and UI Modules

The planner uses a responsive dashboard. Controls appear in a sidebar on desktop and stack above the map on smaller screens.

Module	Responsibility
VehicleSettings	Vehicle count, start/end names, and optional maximum stops per vehicle.
LocationInput	Geocoded search, CSV upload, stop chips, and add-stop interaction.
MapView	OpenStreetMap tiles, depot markers, stop markers, route polylines, and popups.
RouteSummary	Total routes, stops, distance, and estimated time.
RouteComparison	Vehicle-by-vehicle comparison of stops, distance, and time.
RouteCard	Route stops and manual vehicle reassignment.
HistoryPage	Saved route plan summaries from the backend.
Export actions	CSV and PDF downloads from the current plan.

Visual system

- ? Green logistics identity with deep-green actions and mint surfaces.

- ? Custom CSS RouteMaster mark made from route strokes and a waypoint.
- ? Rounded panels, restrained shadows, readable hierarchy, and responsive grids.
- ? Route colors are mapped in the frontend rather than embedded in business logic.

5. Location Input and Geocoding

- ? **Search:** POST /api/geocode calls the backend Nominatim abstraction and displays the selected result.
- ? **Map interaction:** the planner can add a stop interaction with coordinates to the working list.
- ? **CSV upload:** name,address rows are geocoded and valid results are appended.

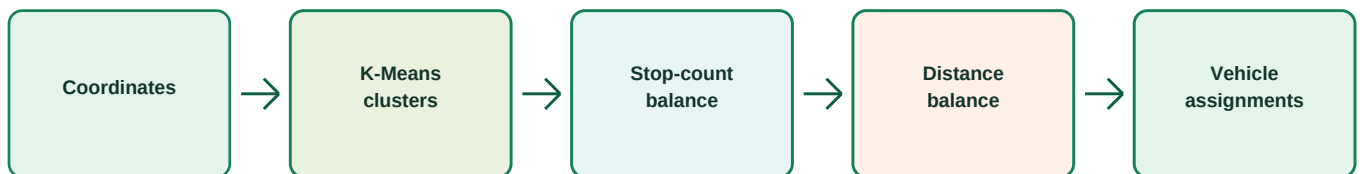
The geocoding service validates empty addresses, uses a User-Agent, applies a request timeout, and converts latitude/longitude values to numbers.

Example CSV

```
name,address
Customer A,Kukatpally Hyderabad
Customer B,Madhapur Hyderabad
Customer C,Gachibowli Hyderabad
```

6. Assignment and Balancing Algorithm

K-Means is used as geographic seeding rather than the complete routing solution. A second pass prevents a large cluster from creating a very long route while another vehicle remains local.



- ? Build one assignment bucket per vehicle.
- ? Estimate a depot-to-stops-to-depot distance using a local geographic approximation.
- ? Move stops from the largest bucket toward the smallest bucket until counts are practical.
- ? Compare longest and shortest estimated routes and evaluate candidate moves.
- ? Try pairwise swaps to reduce distance spread without destroying count balance.
- ? Reject impossible maximum-stop configurations with a clear validation message.

This preserves geographic clustering while making the result more practical for similarly capable vehicles.

7. Route Sequencing and Road Routing

Stage	Implementation
Initial sequence	Nearest-neighbor selection from start through unvisited stops to end.
Improvement	2-opt reverses route segments when Haversine distance becomes shorter.
Road geometry	OSRM receives longitude,latitude pairs and requests full GeoJSON geometry.
Frontend geometry	Backend converts OSRM coordinates to Leaflet [latitude, longitude] pairs.
Metrics	OSRM distance is returned in kilometers and duration in minutes.

If OSRM is unavailable or returns no route, the backend returns an explicit routing error rather than silently presenting a successful road route. The service boundary allows a replacement provider later.

8. Backend Modules

Path	Responsibility
app.py	Creates Flask app, configuration, CORS, database initialization, blueprints, and health endpoint.
config.py	Loads environment values and selects MySQL or SQLite.
routes/geocoding.py	Handles POST /api/geocode and readable errors.
routes/routing.py	Generation/recalculation endpoints, persistence orchestration, response shaping.
routes/history.py	List, read, and delete saved plans.
services/geocoding_service.py	Nominatim address lookup abstraction.
services/clustering_service.py	K-Means seeding plus stop-count and route-distance balancing.
services/optimization_service.py	Haversine distance, nearest neighbor, and 2-opt.
services/road_routing_service.py	OSRM integration and geometry normalization.
models/*	SQLAlchemy entities for plans, locations, routes, and route stops.
utils/validation.py	Shared route payload validation.

9. REST API

Method	Endpoint	Purpose
GET	/api/health	Confirms backend availability.
POST	/api/geocode	Searches an address using Nominatim.
POST	/api/routes/generate	Validates, assigns, optimizes, routes, saves, and returns a plan.
POST	/api/routes/recalculate	Rebuilds route order and geometry after reassignment.
GET	/api/routes	Lists saved route summaries.
GET	/api/routes/	Returns one plan with locations and route stops.
DELETE	/api/routes/	Deletes a saved plan and related records.

Response convention

```
{
  "success": true,
  "routes": [...],
  "summary": {"total_routes": 3, "total_stops": 15, "total_distance": 241.0, "total_duration": 402.0},
  "plan_id": 1
}
```

Errors use the same shape with success=false and a human-readable message.

10. Database Design

The persistence model is normalized around a route plan. A plan owns input locations and generated vehicle routes; route-stop records preserve ordering.

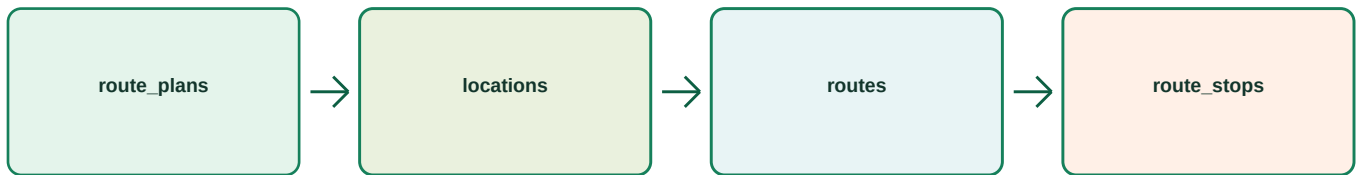


Table	Important fields	Relationship
route_plans	id, start/end coordinates, vehicle_count, total_stops, total_distance, total_duration, created_at	Parent of locations and routes
locations	id, route_plan_id, name, address, latitude, longitude, priority	Belongs to one plan
routes	id, route_plan_id, vehicle_number, distance, duration	Belongs to one plan
route_stops	id, route_id, location_id, stop_order	Connects a location to a route in order

11. Frontend Structure

```

frontend/
??? package.json
??? vite.config.js
??? index.html
??? src/
    ??? main.jsx / App.jsx
    ??? pages/ (LandingPage, PlannerPage, HistoryPage)
    ??? components/ (navigation, controls, map, results)
    ??? services/api.js
    ??? styles.css
  
```

Planner state includes start/end, vehicle count, maximum stops, location list, search results, loading/error state, generated plan, and export/reassignment actions.

12. Export and History

- ? **CSV:** vehicle, stop order, location, address, latitude, and longitude.
- ? **PDF:** RouteMaster heading, summary metrics, vehicle sections, and ordered stops.
- ? **History:** plans are saved in the backend and listed at /history.

History is intentionally unauthenticated, matching the requirement for a local/shared planning tool without accounts.

13. Installation and Operation

Backend setup

```

cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python app.py
  
```

Frontend setup

```

cd frontend
npm install
npm run dev
  
```

Configuration

File	Variables / purpose
frontend/.env	VITE_API_URL=http://localhost:5000
backend/.env	MYSQL_HOST, MYSQL_PORT, MYSQL_USER, MYSQL_PASSWORD, MYSQL_DATABASE

When MYSQL_HOST is not configured, the backend uses local SQLite for a straightforward development demonstration. Configured deployments should use MySQL.

14. Validation and Error Handling

- ? Empty location lists are rejected.
- ? Vehicle count must be positive and cannot exceed stop count.
- ? Missing start/end locations are rejected.
- ? Each stop must include name, latitude, and longitude.
- ? Maximum stop limits are checked against total demand.
- ? Geocoding and routing errors become readable API messages.
- ? Frontend API failures appear in a visible error panel.
- ? External requests use timeouts and service logic stays outside UI components.

15. Testing and Verification

Verification	Result
Frontend production build	Passed with npm run build.
Backend compilation	Passed with Python compileall.
Health endpoint	GET /api/health returned success=true.
15 stops / 3 vehicles	Returned 3 balanced routes with 5/5/5 stop assignment in the validated scenario.
Maximum stops constraint	15 stops with max_stops_per_vehicle=5 returned 3 routes with 5 stops each.
Route generation	Returned geometry, distance, duration, summary, and persisted plan id.

Recommended deployment checks

- ? Set a valid MySQL connection and verify permissions.
- ? Confirm Nominatim and OSRM usage policies for expected traffic.
- ? Use production-grade process managers instead of development servers.
- ? Add automated API tests for invalid payloads and external-service failures.

16. Design Decisions and Extension Points

- ? The routing service is isolated so OSRM can be replaced later.
- ? The assignment layer is isolated from sequencing for future capacity, priority, or time-window constraints.
- ? Route colors are frontend presentation concerns.
- ? SQLAlchemy relationships support richer metadata and auditing later.
- ? The current balancing heuristic is transparent; a future optimization solver could add capacities, service times, or delivery windows.

Current scope is intentionally focused on multi-vehicle distribution, geographic balancing, route sequencing, road geometry, history, editing, and export.

RouteMaster

Plan multiple routes. Move smarter.

This document describes the implemented RouteMaster product architecture and operating model.